

Analog In-Memory Computing for Large Language Model Inference: Opportunities and Challenges

A. Vasilopoulos^{†¶*}, H. Benmeziane[†], J. Büchel[†], W. Simon[†], A. Singh[†], I. Boybat[†], J. Luquin[‡], P. Narayanan[‡], H. Tsai[‡], G. W. Burr[‡], A. Rahimi[†], M. Le Gallo[†], V. Narayanan[§], A. Sebastian^{†*}

[†] IBM Research–Zürich Research Lab, Rüschlikon, Switzerland, [¶] University of Patras, Patras, Greece, * Email: {atv, ase}@zurich.ibm.com

[‡] IBM Research–Silicon Valley Lab, San Jose, CA, USA, [§] IBM Research–T.J. Watson Research Center, Yorktown Heights, NY, USA

Abstract—Recent advancements in large language models (LLMs) have shifted the primary bottleneck of AI hardware from compute to memory capacity and data movement. Analog in-memory computing (AIMC) offers a promising path to address this challenge by enabling matrix-vector multiplication directly within memory arrays, significantly reducing data transfers associated with model weights. In this paper, we discuss the role of AIMC in LLM inference workloads from a holistic systems perspective. We analyze the architecture of modern LLMs and identify which operations are well-suited for AIMC. We further discuss key challenges and opportunities in memory technologies, algorithms, system architecture, and heterogeneous system composition that must be addressed to enable AIMC as a practical accelerator for future AI inference infrastructure.

Index Terms—Analog in-memory computing, 3D memory, AI Inference

I. INTRODUCTION

The rapid evolution of artificial intelligence, owed to the success of large language models (LLMs), has been largely driven by advances in computing hardware, particularly specialized AI accelerators [1]. These advances span both compute and memory systems, including low-precision arithmetic, sparsity-aware execution, and increasingly sophisticated memory hierarchies with large on-chip caches and high-bandwidth memory (HBM). Nevertheless, data movement between memory and compute remains a dominant bottleneck in LLM workloads, both in terms of latency and energy consumption [2].

The structure of LLMs has also kept evolving to curb this issue. Transformer-based architectures dominate current LLMs, evolving to include the mixture-of-experts (MoE) variant [3], [4]. Transformers, however, scale unfavourably with sequence length in memory and compute – quadratically during prefill, linearly during generation – with the memory requirements resulting in large intermediate states that must be stored in system memory, commonly referred to as the KV-cache. Emerging architectures like state-space models (SSMs) and hybrid Transformer-SSM architectures aim to improve efficiency and scalability [5], [6]. Despite their diversity, these models share a common computational structure: they consist of large blocks of parameterized, static computations – matrix-vector multiplications (MVMs) involving model weights – and dynamic sequence-dependent operations, such as attention or state updates. This decomposition, along with examples for each type of operation, appears in Fig. 1.

The relative importance of these components varies across architectures and across different phases of inference, e.g., as a

function of sequence length, but both contribute to the overall system bottlenecks. This decomposition has important implications for hardware design. While static weight computations are dominated by repeated MVM operations with high reuse of model weights across tokens, dynamic components often require frequent and large data movement with minimal reuse.

To curb the aforementioned memory bottleneck in modern accelerators, research has focused on emerging compute paradigms that reduce or circumvent this bottleneck. Analog in-memory computing (AIMC) is a particularly promising non-von Neumann approach, which exploits the physical properties of memory devices to perform operations directly within the memory array, where the data reside. Usually, these operations are limited to MVMs, which dominate computation in modern LLMs [7]. Nevertheless, the dichotomy of static and dynamic computation in modern networks raises a significant question about the role of AIMC in LLM inference. In this paper, we discuss the recent advances in AIMC chips, their role in modern LLM inference, and the challenges and opportunities remaining to position AIMC within future LLM inference infrastructures.

II. RECENT AIMC PROGRESS

AIMC has seen significant progress in recent years, driven by advances in both memory technologies and system architectures. Broadly, AIMC can be categorized based on the underlying memory technology into volatile and non-volatile approaches [7]. The basic structure of an AIMC tile is illustrated in Fig. 2.

Volatile memory-based AIMC has primarily been demonstrated using static random-access memory (SRAM) arrays, leveraging mature CMOS processes and its high-speed operation [8]–[10]. These approaches typically perform computation in the charge or voltage domain, using capacitive accumulation along bitlines (see Fig. 2). These chips offer flexible reprogramming of weights and can support a wide range of workloads and precisions. However, SRAM-based AIMC is fundamentally limited by the large bitcell area and binary weight storage, which constrain the achievable memory density and, consequently, the size of models that can be mapped on-chip, without relying on an external system memory for weight reloading. Emerging alternatives such as gain-cell-based designs aim to improve density relative to SRAM while retaining the flexibility of reprogramming, but

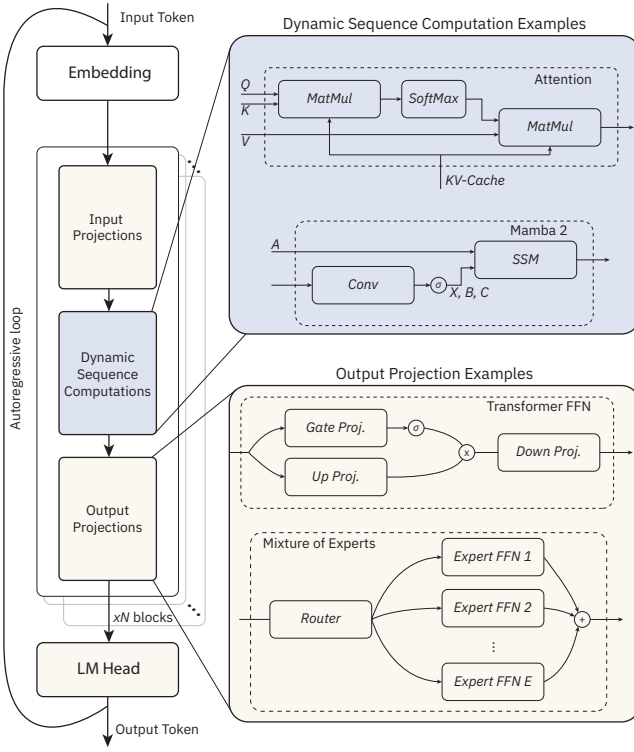


Fig. 1. **The anatomy of the modern LLM architectures.** An LLM consists of an embedding layer, followed by N repeated blocks and a final output head. Each block comprises input projections, a dynamic sequence-dependent operation, and output projections. The dynamic component can correspond to attention mechanisms in Transformer models or state-space updates in SSM-based architectures. The input and output projections are dominated by dense matrix-vector multiplications, including variants such as the conventional feed-forward networks (FFNs) and mixture-of-experts (MoEs). This decomposition is consistent across model families and has direct implications for hardware mapping.

remain at an early stage of development and have not yet demonstrated large-scale AIMC systems [11]. This flexibility enables support for dynamic or frequently changing operands, albeit with significant data movement overhead due to repeated reloading.

Non-volatile memory (NVM)-based AIMC offers a complementary approach, enabling significantly higher on-chip weight density, due to nanoscale device dimensions and inherent multi-level storage capabilities [12]. A wide range of devices have been explored for AIMC, including phase-change memory (PCM) [13], [14], resistive random-access memory (RRAM) [15], magnetoresistive random-access memory (MRAM) [16], and Flash [17], [18], showing promising results in terms of compute density and efficiency. However, a key characteristic of NVM-based AIMC is the strong asymmetry between read and write operations. While reads can be performed rather efficiently and in parallel across large arrays, programming memory cells to analog states typically requires iterative read-write verify procedures that lead to significantly higher energy and latency costs.

As a result, NVM-based AIMC architectures are naturally suited to weight-stationary operations, where model param-

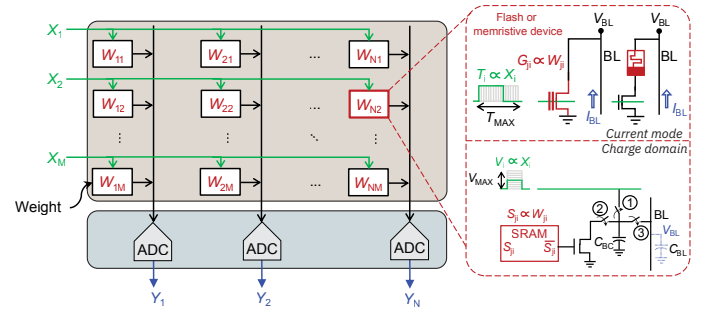


Fig. 2. **Organization of an AIMC tile.** Weights are stored in memory elements (volatile or non-volatile), while inputs are applied in the analog domain (e.g., voltage or time encoding). Their interaction within the array results in analog accumulation along the bitlines, implementing a MVM. The accumulated signals are then digitized by ADCs to produce the output. Both current-mode (NVM-based) and charge-domain (SRAM-based) realizations are illustrated.

eters are reused extensively during inference. On the other hand, volatile-memory-based AIMC allows frequent weight updates and reloading, enabling more flexible execution at the cost of reduced density. These complementary properties highlight a fundamental trade-off between density and programmability in current AIMC systems, which directly determines their suitability for the different components of LLM workloads.

III. ROLE OF AIMC IN LLM INFERENCE

In the context of LLM inference, the decomposition into static and dynamic operations maps naturally to the characteristics of volatile and non-volatile AIMC implementations. Static computations, consisting of large MVMs with fixed weights during inference, are shared across all tokens and exhibit high reuse. This makes them a natural fit for NVM-based AIMC, where weights can be programmed once and reused across many inferences [19], [20]. Volatile memory-based AIMC also offers advantages in this context, but is constrained by the amount of on-chip storage – GBs of weights are required for even small-scale LLMs – and typically relies on reloading weights from external memory, similar to cache hierarchies in conventional digital accelerators.

In contrast, the dynamic components of LLM inference pose significant challenges for AIMC [21]. In Transformer-based models, attention involves operations with dynamically changing operands. During prefill, this corresponds to matrix-matrix multiplications, while during decoding, to matrix-vector multiplications between the query vector and the KV-cache. In both cases, the operands depend on the input sequence and therefore cannot be pre-programmed in NVM arrays. Moreover, the KV-cache has limited reuse across tokens and users, making it impractical to amortize the cost of programming the matrices into the memory devices. Importantly, the size of the KV-cache grows with sequence length, causing the dynamic component of the workload to scale accordingly, while the static weight computations remain constant. As a result, the fraction of operations that can be effectively accelerated by

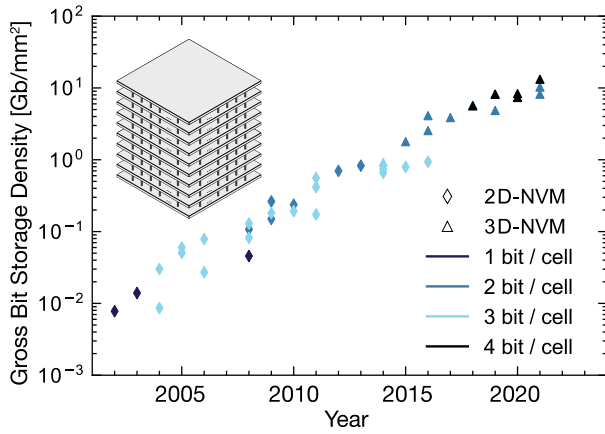


Fig. 3. **NVM density scaling.** In recent years, the continuous scaling of NVM density has been driven by 3D-enabled memory and continuous higher multi-level capabilities of each cell, with the key contributor being 3D NAND Flash memory.

AIMC decreases with increasing sequence length, as dynamic attention computations become dominant.

Volatile memory-based AIMC can, in principle, support these dynamic operations through frequent weight reloading. However, the low reuse of dynamic data, particularly the KV-cache, results in an operating model where data movement dominates execution time and energy, effectively reverting the system to a von Neumann-like behavior.

For SSM-based architectures, the nature of dynamic computation differs. Instead of attention, these models rely on state update and scan-like operations, which are not expressed as dense MVMs and are therefore less amenable to acceleration with current AIMC primitives. Nevertheless, these operations typically exhibit more favorable scaling with sequence length compared to attention, reducing their relative contribution to the overall workload as sequence length increases.

These observations highlight two key requirements for effectively leveraging AIMC in LLM inference. First, the workload must be dominated by static, high-reuse MVMs, where the benefits of in-memory computation can be fully exploited. Second, sufficient on-chip memory density is required to store these weights without frequent reloading or reprogramming. Together, these conditions define the regimes in which AIMC can provide substantial system-level benefits.

IV. CHALLENGES AND OPPORTUNITIES

As discussed in the previous section, on-chip memory density is a key requirement to unlock the efficiency of AIMC. This is a fundamental advantage of NVM-based approaches, which combine small device footprints and multi-level storage with the ability to scale in the third dimension. In particular, 3D integration has enabled continuous growth in storage density over the past decade, as illustrated in Fig. 3.

While 3D memory technologies significantly increase storage capacity (Gb/mm²), they do not directly scale compute

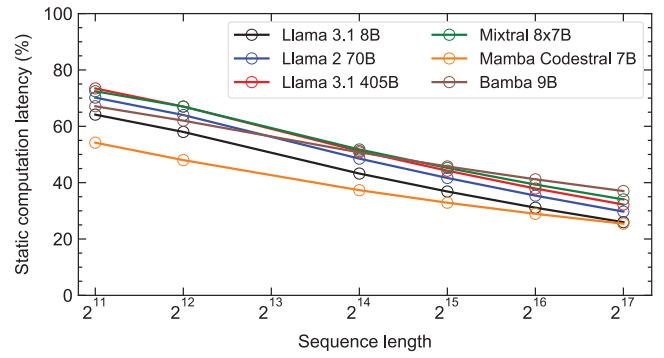


Fig. 4. **Fraction of static computation in LLM inference.** The proportion of computation associated with static MVMs decreases with increasing sequence length, reducing the fraction of the workload that can be efficiently accelerated by AIMC. Results are obtained by profiling inference latency for generating up to different sequence lengths on representative LLM architectures using 8 A100 NVIDIA GPUs.

density (TOPS/mm²), as computation is typically performed within individual 2D slices. However, the advantage of increased density still offers advantages at the system level. By mapping model parameters across multiple tiers, different portions of the model can be accessed sequentially with minimal data movement, effectively enabling a form of low-cost weight reload without relying on external memory transfers. In this sense, 3D AIMC architectures can be viewed as combining the density advantages of non-volatile memory with some of the flexibility of volatile approaches.

This capability is particularly beneficial for modern LLMs, where model sizes exceed the capacity of conventional on-chip memories. In [22], we studied LLMs inference on 3D AIMC hardware and observed significant energy efficiency improvements, especially for MoE-based architectures – up to 3 orders of magnitude – which can exploit increased capacity without proportionally increasing computation.

Another long-standing question for AIMC systems is the achievable inference accuracy. In both volatile and non-volatile approaches, accuracy can be scaled by pooling more devices together to represent a single weight. Ideally for the NVM-based approaches, though, the aim is to store weights directly in analog form, mapping each weight to a single device (or to a pair of differential devices). The noise stemming from the devices and the analog computation, however, degrades accuracy and creates a stochastic execution model. In recent works, we have shown that analog hardware-aware training [23] and post-training adaptation [24] can effectively mitigate these effects, suggesting that accuracy is not the fundamental barrier to deploying AIMC for LLM inference.

However, even with sufficient memory density and acceptable accuracy, the system-level benefits of AIMC remain fundamentally limited by the dynamic components of LLM inference. As illustrated in Fig. 4, the fraction of computation associated with static operations decreases as sequence length increases, with dynamic computations eventually dominating

the workload. This observation has direct implications for system design. Rather than attempting to accelerate the entire workload using a single architecture, it may be more effective to adopt a heterogeneous approach, where AIMC accelerators target the static, high-reuse components, and digital processors handle the dynamic, sequence-dependent operations.

Previous efforts on heterogeneous architectures have integrated analog and digital compute monolithically within the same chip and often introduced trade-offs, limiting either the model size or the supported sequence length. In contrast, decoupling these components enables each to be optimized independently. An AIMC accelerator can be designed to maximize storage density and efficiently execute large static MVMs, while a digital accelerator can be tailored to handle attention and KV-cache operations at the sequence lengths of the target applications.

This separation enables flexible deployment across application domains. In edge scenarios with shorter sequence lengths, AIMC accelerators can be paired with lightweight digital processors to handle dynamic operations. In data center environments, multiple users can share statically programmed weights on AIMC hardware, while high-performance digital accelerators manage dynamic workloads such as attention and KV-cache updates. More broadly, offloading model parameters from conventional memory hierarchies to AIMC enables more efficient execution of dynamic workloads on digital accelerators, while simultaneously improving efficiency for the static components of inference.

V. CONCLUSIONS

AIMC offers a compelling path to address the memory bottlenecks of modern LLM inference by efficiently accelerating static, high-reuse MVMs. However, its benefits are inherently limited by the dynamic components of these workloads, which become increasingly dominant at longer sequence lengths. As a result, AIMC is unlikely to serve as a standalone solution for LLM inference, but rather as a specialized co-processor within heterogeneous inference systems. In data center deployments, this separation enables more efficient inference through the use of the highly efficient AIMC for the static computation and the better utilization of digital accelerators by decoupling parameter storage from dynamic data. At the edge, AIMC can enable larger models within constrained power and area budgets by providing dense on-chip weight storage. Recent advances in 3D memory integration and analog-aware training techniques further strengthen the viability of this approach by addressing key challenges in density and accuracy. Nevertheless, fully realizing the potential of AIMC requires continued research at the intersection of devices, architectures, and systems, particularly to quantify its system-level value proposition and to identify the most suitable memory technologies for large-scale deployment.

ACKNOWLEDGMENT

We would like to thank J. Burns and M. Khare for managerial support.

REFERENCES

- [1] A. Reuther *et al.*, “AI and ML Accelerator Survey and Trends,” in *IEEE High Performance Extreme Computing Conference (HPEC)*, Sep. 2022.
- [2] N. P. Jouppi *et al.*, “Ten Lessons From Three Generations Shaped Google’s TPUv4i: Industrial Product,” in *International Symposium on Computer Architecture (ISCA)*, Jun. 2021.
- [3] A. Vaswani *et al.*, “Attention is all you need,” in *International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2017.
- [4] W. Fedus *et al.*, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [5] T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” 2024.
- [6] H. Face, “Bamba: Inference-efficient hybrid Mamba2 model,” 2024, <https://huggingface.co/blog/bamba>.
- [7] A. Singh *et al.*, “The design of analogue in-memory computing tiles,” *Nature Electronics*, vol. 8, no. 12, pp. 1156–1169, Dec. 2025.
- [8] C.-J. Jhang *et al.*, “Challenges and Trends of SRAM-Based Computing-In-Memory for AI Edge Devices,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 5, pp. 1773–1786, May 2021.
- [9] Z. Jiang *et al.*, “C3SRAM: An In-Memory-Computing SRAM Macro Based on Robust Capacitive Coupling Computing Mechanism,” *IEEE Journal of Solid-State Circuits*, vol. 55, no. 7, pp. 1888–1897, Jul. 2020.
- [10] J. Lee *et al.*, “A Switched-Capacitor SRAM In-memory Computing Macro with High-precision, High-efficiency Differential Architecture,” in *European Solid-State Electronics Research Conference (ESSERC)*, 2024.
- [11] N. Leroux *et al.*, “Analog in-memory computing attention mechanism for fast and energy-efficient large language models,” *Nature Computational Science*, vol. 5, no. 9, pp. 813–824, Sep. 2025.
- [12] M. Lanza *et al.*, “The growing memristor industry,” *Nature*, vol. 640, no. 8059, pp. 613–622, Apr. 2025.
- [13] S. Ambrogio *et al.*, “An analog-AI chip for energy-efficient speech recognition and transcription,” *Nature*, vol. 620, no. 7975, pp. 768–775, Aug 2023.
- [14] M. Le Gallo, R. Khaddam-Aljameh, M. Stanisavljevic, A. Vasilopoulos, B. Kersting, M. Dazzi, G. Karunaratne, M. Brändli, A. Singh, S. M. Mueller *et al.*, “A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference,” *Nature Electronics*, vol. 6, no. 9, pp. 680–693, 2023.
- [15] W. Wan *et al.*, “A compute-in-memory chip based on resistive random-access memory,” *Nature*, vol. 608, no. 7923, pp. 504–512, Aug 2022.
- [16] P. Deaville *et al.*, “A Fully Row/Column-Parallel MRAM in-Memory Computing Macro With Memory-Resistance Boosting and Weighted Multi-Column ADC Readout,” *IEEE Journal of Solid-State Circuits*, vol. 60, no. 5, pp. 1856–1866, May 2025.
- [17] W. Shim *et al.*, “Technological Design of 3D NAND-Based Compute-in-Memory Architecture for GB-Scale Deep Neural Network,” *IEEE Electron Device Letters*, vol. 42, no. 2, pp. 160–163, Feb. 2021.
- [18] H.-T. Lue *et al.*, “Introduction of 3D AND-type Flash Memory and Its Applications to Computing-in-Memory (CIM),” in *International Symposium on VLSI Technology, Systems and Applications (VLSI-TSA)*, 2021.
- [19] A. Chen *et al.*, “Demonstration of transformer-based ALBERT model on a 14nm analog AI inference chip,” *Nature Communications*, vol. 16, no. 1, Sep. 2025.
- [20] J. Büchel *et al.*, “Kernel approximation using analogue in-memory computing,” *Nature Machine Intelligence*, vol. 6, no. 12, pp. 1605–1615, Dec. 2024.
- [21] H. Tsai *et al.*, “Analog AI Accelerators for Transformer-based Language Models: Hardware, Workload, and Power Performance,” in *International Memory Workshop (IMW)*, May 2025.
- [22] J. Büchel *et al.*, “Efficient scaling of large language models with mixture of experts and 3D analog in-memory computing,” *Nature Computational Science*, vol. 5, no. 1, pp. 13–26, Jan. 2025.
- [23] J. Büchel *et al.*, “Analog foundation models,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [24] Y. Hou *et al.*, “NORA: Noise-Optimized Rescaling of LLMs on Analog Compute-in-Memory Accelerators,” in *Design, Automation & Test in Europe Conference (DATE)*, 2025.